

Introducing Test-Driven Modeling to MERODE through a Domain-Specific Testing Language and Automated Model Validation

Online Supplementary Material

Dragana Sunaric^[0009–0004–0115–0817] and
Charlotte Verbruggen^[0000–0003–0418–2633] and
Dominik Bork^[0000–0001–8259–2297]

TU Wien, Business Informatics Group, Erzherzog-Johann-Platz 1, 1040 Vienna,
Austria {charlotte.verbruggen,dominik.bork}@tuwien.ac.at

Abstract. In this document, the online supplementary material is provided for the paper *Introducing Test-Driven Modeling to MERODE through a Domain-Specific Testing Language and Automated Model Validation*, submitted to POEM 2026.

Keywords: Test-Driven Development · Test-Driven Modeling · MERODE.

1 Design and Implementation

1.1 System Architecture

The proposed solution extends the existing MERLIN ecosystem with automated test-driven modeling capabilities while preserving the existing architecture of the toolchain. Rather than integrating the execution logic directly into MERLIN, the testing functionality is realized as an independent backend service that communicates with the existing MERLIN Frontend through a REST interface. This design minimizes changes to the MERLIN code and allows the testing engine to evolve independently from the modeling environment.

The overall architecture consists of three main components: the MERLIN Frontend, the MERLIN Backend, and the MUnit Interpreter Backend. Figure 1 illustrates the overall architecture of the proposed solution and the interactions between its three main components.

The MERLIN Frontend remains the primary user interface for creating and editing MERODE models. As part of this work, it has been extended with a dedicated editor for writing MUnit test scripts and functionality for executing these tests and visualizing the resulting test report.

The MERLIN Backend continues to provide persistence and version management for modeling projects. Besides storing MERODE models, it is extended to persist MUnit test scripts together with their corresponding models. Therefore,

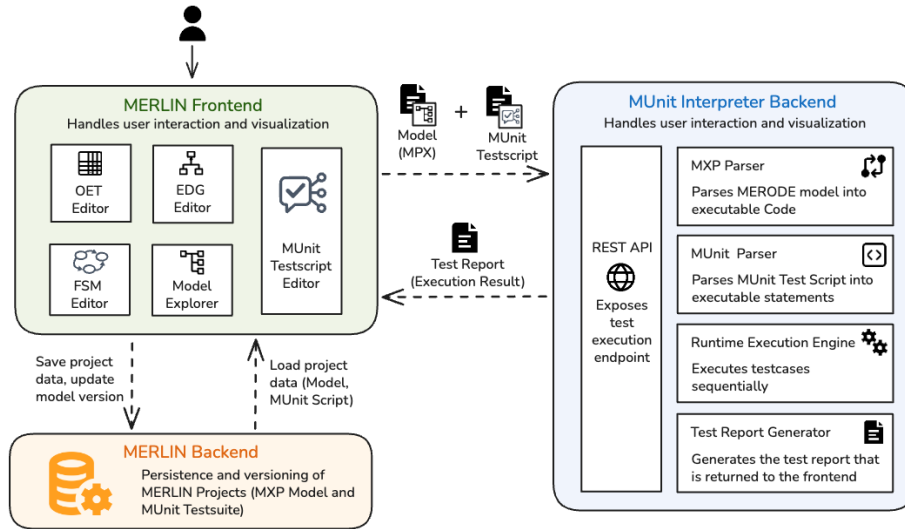


Fig. 1: High-level system architecture diagram

MUnit test files are included into the versioning capabilities of the MERLIN backend.

The main contribution of this thesis is the MUnit Interpreter Backend, implemented as an independent Spring Boot application. It exposes a REST API that accepts a MERODE model together with an MUnit test specification, executes the test suite, and returns a structured test report to the frontend.

Communication between the components follows a synchronous request-response pattern. The MERLIN Frontend interacts with the MERLIN Backend to load and store project data, test execution requests are sent directly to the MUnit Interpreter Backend. Each execution request contains the exported MPX model and the associated MUnit test script. After processing the request, the interpreter returns a structured test report that is presented to the user within the MERLIN Frontend.

Integration into the MERODE Toolchain The primary objective of this work is to integrate automated model testing directly into the modeling workflow. Instead of requiring users to export their model and generate a prototype application, test cases can now be written and executed directly from within the MERLIN editor. This enables immediate feedback while modeling and reduces the effort required to validate model correctness.

Prior to this work, MERLIN already provided a web-based environment for creating and editing MERODE models. Models are persisted by the MERLIN Backend, which maintains complete project versions and allows users to continue their work across editing sessions. Furthermore, MERLIN supports exporting

models in the MPX format, which serves as the interchange format between the modeling environment and external tools.

The integration presented in this thesis builds directly upon these existing capabilities. Rather than modifying the internal representation of MERODE models or extending the existing backend with execution logic, an independent MUnit Interpreter Backend is introduced. As shown in Figure 1, the interpreter operates exclusively on exported MPX models and MUnit test scripts, allowing it to execute automated tests without requiring direct access to MERLIN’s internal implementation. This approach minimizes the impact on the existing code base while providing a clear separation between model management and test execution.

The MUnit Interpreter builds upon parts of the existing MERLIN Prototyper (Code Generator). Instead of generating a complete executable prototype application, however, the code generation process was adapted specifically for automated testing. Only the classes required for model execution are generated, while user interface components and other unnecessary artifacts are omitted. These optimizations significantly reduce generation time and improve the overall performance of test execution.

Only limited extensions to the existing MERLIN components were required. As highlighted in Figure 1, the MERLIN Frontend was extended with an editor for writing MUnit test scripts, a mechanism for executing test suites, and a user interface for displaying the resulting test report. On the persistence layer, the MERLIN Backend was enhanced to store and version MUnit test scripts together with their corresponding MERODE models. Consequently, test specifications become first-class project artifacts that are managed alongside the models they validate.

Overall, the integration preserves the original responsibilities of the existing MERLIN components. As summarized in Figure 1, the MERLIN Frontend remains responsible for user interaction and visualization, while the MERLIN Backend continues to manage persistence and project versioning. The newly introduced MUnit Interpreter Backend is solely responsible for automated test execution, thereby maintaining a clear separation between model management and runtime execution.

The MUnit Interpreter The MUnit Interpreter Backend constitutes the core technical contribution of this thesis. It is responsible for executing MUnit test suites against MERODE models and is implemented as an independent Spring Boot application. Its internal components, shown on the right-hand side of Figure 1, are organized according to the different stages of the test execution process.

The **REST API** serves as the entry point to the interpreter. It exposes a `POST /run` endpoint that receives an exported MPX model together with an MUnit test specification from the MERLIN Frontend. The API validates incoming requests, invokes the execution pipeline, and returns the resulting test report as a structured JSON response. The **MPX Parser** is responsible for processing the exported MERODE model. It parses the MPX representation

and invokes an adapted version of the existing MERLIN Prototyper code generation process to construct an executable representation of the model. Unlike the original code generator, which produces a complete prototype application, the interpreter generates only the artifacts required for automated test execution, thereby reducing generation time and improving runtime performance. The **MUnit Parser** processes the textual test specification. It is implemented using ANTLR and transforms the input script into an Abstract Syntax Tree (AST), which serves as the internal representation of the test suite. During this process, the parser performs syntactic validation and reports grammar errors before test execution begins. The **Runtime Execution Engine** coordinates the execution of the parsed test suite. For each test case, it initializes the execution environment by resetting the Information Base, executes the test statements in sequence, invokes the generated model operations, and evaluates all assertions. The engine therefore acts as the central orchestrator of the interpreter. After execution has completed, the **Test Report Generator** aggregates the execution results into a structured report. For each executed test case, it records whether the test passed or failed, captures occurring error messages, and collects additional execution information required for presentation in the MERLIN Frontend. The generated report is finally returned to the frontend as a JSON response.

1.2 Test DSL Design

This section presents the design of MUnit, the domain-specific language developed for specifying automated tests of MERODE models. The section first describes the concrete syntax for users of MUnit, followed by the formal grammar specification for the language and the Abstract Syntax Tree (AST) representation which is used internally by the MUnit Interpreter during parsing.

Concrete Syntax The MUnit language was designed as a textual external domain-specific language for specifying automated tests for MERODE models. Its syntax follows the style of modern unit-testing frameworks and imperative programming languages to provide a familiar and readable notation for users. Rather than introducing a completely new syntax, MUnit adopts common programming concepts such as variables, method invocations, and assertions while extending them with MERODE-specific constructs for interacting with conceptual models.

An MUnit test suite consists of one or more **test** blocks, each representing an independent test case. Test cases are executed sequentially by the interpreter, while each individual test is evaluated in isolation. Within a test case, users may declare variables, create MERODE object instances, invoke model events, assign values to local variables, and verify expected behaviour using assertions. This structure closely resembles established unit-testing frameworks, allowing model tests to be written in a concise and intuitive manner.

The language supports two complementary categories of tests. Behavioural tests validate the dynamic behaviour of a MERODE model by creating object

instances, executing events, and verifying state transitions or attribute values. Structural tests, on the other hand, verify static properties of the model itself, such as the existence of object types, methods, finite state machines, states, or attributes. Together, these two categories allow both the structural correctness and the behavioural semantics of a MERODE model to be validated automatically.

Listing 1.1 illustrates a simple behavioural test case. The example creates a `Patient` object using the generated creation event, stores the resulting object reference in a local variable, and verifies that the newly created object has entered the expected `exists` lifecycle state.

Listing 1.1: Example MUnit test case

```

1  test testIfPatientExistsAfterCreation {
2      Patient p = mecrpatient();
3      assert p.getState().getName() == "exists";
4  }
```

Grammar Definition The concrete syntax of MUnit is formally specified using a context-free grammar implemented in ANTLR (Another Tool for Language Recognition). ANTLR was selected because it provides a concise way of defining the language syntax while automatically generating both a lexer and parser for Java, the implementation language of the MUnit Interpreter.

The grammar defines the syntactic structure of all valid MUnit test suites, including test case declarations, statements, assertions, variable declarations, and expressions. During test execution, the generated parser validates the input script against this grammar before constructing an Abstract Syntax Tree (AST). Syntax errors are therefore detected before any test execution begins, preventing malformed test specifications from entering the execution pipeline.

Figure ?? illustrates the role of the grammar within the interpreter. A textual MUnit test suite is first tokenized by the ANTLR lexer and subsequently parsed according to the grammar. The resulting Abstract Syntax Tree serves as the input for the Runtime Execution Engine.

Abstract Syntax Model After successful parsing, the textual representation of an MUnit test suite is transformed into an Abstract Syntax Tree (AST). Whereas the parse tree generated by ANTLR closely reflects the concrete grammar, the AST provides a simplified representation that captures only the semantic information required during execution. Consequently, the Runtime Execution Engine operates exclusively on the AST rather than directly traversing the parser-specific data structures.

Figure 2 presents the class hierarchy of the implemented AST. The model is organized around a small number of abstract interfaces that represent the different categories of language constructs. Concrete node classes specialize these interfaces and represent the individual elements of the MUnit language.

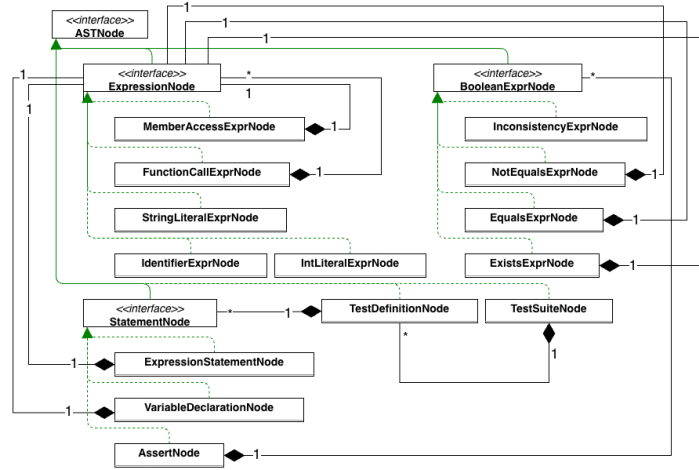


Fig. 2: Class hierarchy of the Abstract Syntax Tree used by the MUnit Interpreter.

The root of every parsed test specification is represented by the `TestSuiteNode`, which contains the complete test suite. A test suite consists of one or more `TestDefinitionNode` instances, each representing an individual `test` block from the source file. Every test definition contains an ordered sequence of statements that are executed sequentially by the Runtime Execution Engine.

Statements are represented through the `StatementNode` interface, which is specialized by the three executable statement types supported by the language.

`VariableDeclarationNode` represents variable declarations, `ExpressionStatementNode` models standalone expressions such as method invocations, and `AssertNode` represents assertions that are evaluated during test execution.

Expressions are represented independently through the `ExpressionNode` hierarchy. This hierarchy contains the language constructs required to access runtime values, including identifiers, literal values, member access expressions, and function calls. Separating expressions from executable statements simplifies evaluation, as expressions can be reused throughout the language without introducing duplicate execution logic.

Boolean conditions used by assertions are represented by the dedicated `BooleanExprNode` hierarchy. The implementation distinguishes between comparison expressions, such as equality and inequality checks, structural assertions using the `exists` operator, and consistency checks that validate properties of the MERODE model.

The AST is constructed using the visitor pattern. During parsing, the ANTLR-generated parse tree is traversed by a dedicated `ASTBuilder`, which creates the corresponding AST node for each grammar production. This transformation decouples the execution engine from ANTLR-specific parser classes, resulting in a

cleaner architecture and allowing future modifications to the grammar without affecting the runtime implementation.

Compared to directly executing the ANTLR parse tree, the AST provides a more compact and semantically meaningful representation of the test suite. Since only information relevant for execution is retained, the Runtime Execution Engine can operate on a simplified object model that is independent of the concrete parser implementation, improving maintainability and facilitating future extensions of the MUnit language.

1.3 Frontend Design

To integrate automated model testing into the existing MERLIN environment, the frontend was extended with a dedicated testing interface. The implementation focuses on three complementary aspects: editing MUnit test scripts, executing test suites, and presenting execution results directly within the modeling environment. By integrating these capabilities into the existing MERLIN interface, users can create, execute, and refine model tests without leaving the modeling environment.

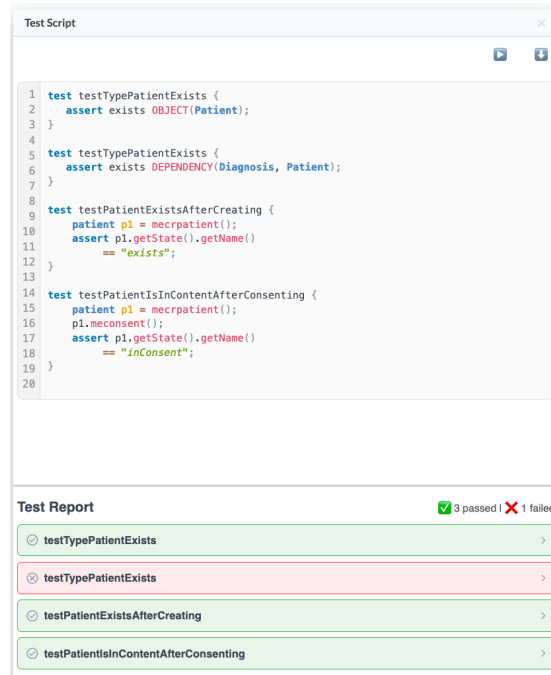


Fig. 3: Integrated MUnit testing interface within the MERLIN Frontend. The upper part contains the MUnit editor, while the lower part displays the generated test report after execution.

Figure 3 shows the integrated MUnit test editor that was added to the existing MERLIN Frontend, allowing test cases to be managed together with their corresponding MERODE models. Consequently, every project contains both the conceptual model and its associated MUnit test suite, which are loaded and stored together by the MERLIN Backend.

The editor provides a lightweight development environment for writing and modifying MUnit test specifications. To improve readability and usability, it supports syntax highlighting and line numbering. The editor additionally provides dedicated actions for exporting the current test suite and executing it directly through the MUnit Interpreter Backend.

For each failed test case, the corresponding report entry is highlighted in red and can be expanded to reveal detailed diagnostic information, an example is illustrated in Figure 4. To assist users in locating the source of the error, the expanded view includes the type of failure, a descriptive error message, and additional execution details, such as the assertion that caused the failure.

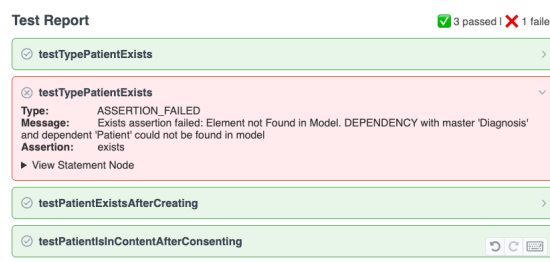


Fig. 4: Example of the integrated test report showing a failed test case with expanded diagnostic information.

This integrated presentation of editing and execution results establishes a short edit–execute–debug cycle. Users can directly modify either the MERODE model or the MUnit test specification based on the reported feedback and immediately re-execute the test suite. As a result, automated testing becomes a natural part of the existing MERLIN modeling workflow rather than a separate development activity.

2 Evaluation

2.1 Study Design

To evaluate the proposed testing framework, a qualitative user study was conducted with experts involved in teaching MERODE. The study aimed to investigate the usability, expressiveness, and perceived educational value of the proposed MUnit language and its integration into the MERLIN environment. Participants completed a series of practical exercises using the testing extension, followed by a questionnaire assessing different aspects of the developed solution.

Participants The evaluation involved five participants who are actively involved in teaching MERODE at three European universities. Three participants were affiliated with KU Leuven, one with ZHAW Zurich University of Applied Sciences, and one with the University of Namur.

All participants had prior experience with MERODE. Three participants reported being very familiar with MERODE (6/6), one reported a familiarity of 5/6, and one participant reported 3/6. Furthermore, every participant had at least three years of experience teaching or working with MERODE, with three participants reporting more than five years of experience.

The participants also possessed varying levels of experience with automated testing. Self-assessed familiarity ranged from 2/6 to 6/6, ensuring that both experienced and less experienced users of automated testing were represented. Four of the five participants had previously worked with Test-Driven Development (TDD) or Test-Driven Modeling (TDM), whereas one participant had no prior experience with either approach. Table 1 summarizes the demographic characteristics of the participants.

Table 1: Demographic characteristics of the study participants

Characteristic	Distribution
Number of participants	5
Institution	KU Leuven (3) ZHAW Zurich University of Applied Sciences (1) University of Namur (1)
Teaching experience	3–5 years (2) More than 5 years (3)
Experience with MERODE	3–5 years (3) More than 5 years (2)
MERODE familiarity (1–6)	Mean = 5.2 Median = 6
Automated testing familiarity (1–6)	Mean = 4.4 Median = 5
Previous TDD/TDM experience	Yes (4) No (1)

Procedure Each evaluation session lasted approximately 30 minutes and was conducted individually using a remotely hosted instance of the extended MERLIN environment. Since the implementation was deployed on the author’s local machine, participants accessed the software through Google Chrome Remote Desktop while retaining full control of the application.

At the beginning of each session, participants received a brief introduction to the purpose of the study and the testing extension. The concepts of test suites, independent test execution, and the execution model of the interpreter were ex-

plained before the practical exercises commenced. Participants were encouraged to ask questions whenever something was unclear.

During the practical exercises, participants were asked to think aloud whenever possible. They were encouraged to verbalize their expectations, reasoning, and any difficulties encountered while interacting with the tool. This qualitative feedback complemented the questionnaire responses and provided additional insights into the intuitiveness of the developed testing framework.

After completing all exercises, participants filled out an online questionnaire assessing the usability, expressiveness, and educational value of the testing extension. The questionnaire was completed immediately after the practical session to minimize recall bias. Follow-up discussions were conducted where additional clarification was required.

Tasks The practical part of the evaluation consisted of four guided exercises that progressively introduced participants to the functionality of the proposed testing framework. Each exercise focused on a different aspect of the MUnit language and the integrated testing environment while building upon the knowledge acquired in previous exercises. An overview of the exercises and their respective learning objectives is provided in Table 2.

Rather than asking participants to write test suites from scratch, every exercise provided either a partially completed test case or an intentionally failing artifact. This allowed participants to focus on understanding the language constructs and the feedback provided by the tool instead of spending time learning the syntax. Throughout the exercises, participants were encouraged to think aloud and verbalize their reasoning whenever possible.

Table 2: Overview of the tasks and their objectives

Exercise	Objective	Evaluated Concepts
1	Testing existence dependencies	Structural assertions, DSL syntax, existence dependencies
2	Testing object creation and behaviour	Object creation, event execution, FSM state transitions, behavioural assertions
3	Correcting an incorrect MERODE model	Interpretation of test reports, debugging, model refinement
4	Correcting an incorrect MUnit test	DSL understanding, debugging, interpretation of execution feedback

Exercise 1 – Testing Existence Dependencies The first exercise introduced participants to structural assertions. Participants were given a completed example verifying the existence dependencies of one object type and were then asked to create a similar test case for another object. The objective was to familiarize

participants with the syntax of MUnit and the use of assertions for validating structural properties of a MERODE model.

Exercise 2 – Testing Object Creation and Behaviour The second exercise focused on behavioural testing. Based on an existing example, participants created a new test case that instantiated model objects, executed domain events, and verified the resulting object states. This exercise introduced the concepts of object creation, event execution, and state assertions while demonstrating how behavioural aspects of a MERODE model can be validated automatically.

Exercise 3 – Correcting an Incorrect MERODE Model The third exercise shifted the focus from writing tests to interpreting test results. Participants received a MERODE model containing modelling errors together with a test case describing the intended behaviour. Their task was to execute the test suite, analyse the reported failures, identify the modelling errors, and modify the MERODE model until all assertions passed. This exercise evaluated whether the generated test reports provide sufficient information to support debugging and model refinement.

Exercise 4 – Correcting an Incorrect Test Case The final exercise reversed the previous scenario. Participants were provided with a correct MERODE model but an intentionally incorrect MUnit test case. Their task was to identify the mistakes in the test specification itself and modify the test until it correctly validated the model. This exercise assessed the understandability of the language syntax as well as the participants' ability to interpret execution feedback and distinguish between modelling errors and incorrectly specified tests.

2.2 Data Collection

The questionnaire consisted of six thematic sections addressing both the technical and educational aspects of the proposed approach. Besides collecting demographic information, the questionnaire assessed the usability and expressiveness of the MUnit language, the perceived educational value of automated test-driven modeling, and the suitability of integrating the proposed framework into MERODE courses. Most questions were formulated as statements and answered using a six-point Likert scale ranging from *Strongly agree (1)* to *Strongly disagree (5)*. In addition, participants were given the opportunity to provide qualitative feedback through several open-ended questions. Table 3 provides an overview of the questionnaire structure.

In addition to the questionnaire responses, qualitative observations were recorded throughout each evaluation session. While participants worked on the practical exercises, comments, questions, and notable interactions with the testing environment were noted. The observational notes primarily focused on identifying syntax misunderstandings, clarification questions raised by participants,

Table 3: Questionnaire statements and abbreviations used in the result figures.

Short Label	Questionnaire Statement
Usability	
DSL syntax	The syntax of the domain specific language for writing the test cases was easy to understand.
Writing test cases	Writing test cases was straightforward.
Feedback messages	The feedback messages were understandable.
Learning the tool	I could quickly learn how to use the tool.
Testing workflow	The testing workflow felt intuitive.
Expressiveness	
DSL expressiveness	The DSL allowed me to express relevant MERODE test scenarios.
EDG testing	The DSL adequately supports testing the EDG structure.
FSM testing	The DSL adequately supports testing FSM transitions.
Model behaviour	I was able to express expected model behavior clearly.
Educational Value	
Understanding MERODE	Automated testing could improve students' understanding of MERODE.
Immediate feedback	The immediate feedback given by this tool supports learning.
Early error detection	Test-driven modeling could help students detect modeling errors earlier.
Model quality	The approach could improve model quality in educational settings.
Course Integration	
MERODE exercises	I could imagine integrating this tool into MERODE exercises.
Teaching workflow	The tool fits naturally into existing MERODE teaching workflows.
Assignment evaluation	The tool could support assignment evaluation.
Student engagement	The tool could improve student engagement.

and suggestions for improving both the MUnit language and the integrated testing environment. Particular attention was given to situations in which participants hesitated, misinterpreted language constructs, or experienced difficulties understanding execution feedback.

In addition to the questionnaire responses and observational notes, a set of objective performance metrics was collected throughout the practical exercises. The collected metrics focused on the efficiency and correctness of task completion. Table 4 summarizes the objective measures recorded during the evaluation.

2.3 Results

Questionnaire Results The questionnaire results indicate that participants perceived the MUnit testing environment as easy to learn and use. As shown in Figure 5, all usability-related statements received positive ratings, with mean scores ranging from 1.4 to 3.0 on a five-point Likert scale (1 = Strongly agree, 5 = Strongly disagree). The statement *"I could quickly learn how to use the tool"* received the strongest agreement (mean = 1.4), suggesting that participants were able to become familiar with the testing workflow after only a short introduction.

Table 4: Objective metrics collected during the evaluation

Metric	Description
Task Completion Rate	Whether participants successfully completed each exercise without assistance.
Completion Time	The time required to complete each exercise, measured from the beginning of the task until its successful completion.
Number of Execution Attempts	The number of times participants executed a test suite before successfully completing the exercise.
Number of Syntax Errors	The number of syntax-related errors encountered while creating or modifying MUnit test cases.

Similarly, participants agreed that writing test cases was straightforward (mean = 1.6) and that the DSL syntax was easy to understand (mean = 2.0).

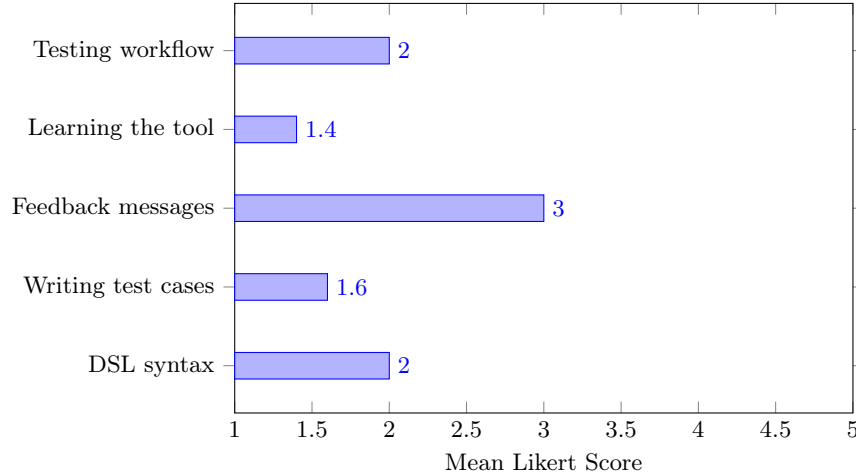


Fig. 5: Mean usability ratings (1 = Strongly agree, 5 = Strongly disagree). The abbreviated labels are listed in 3

The testing workflow was also perceived as intuitive (mean = 2.0), indicating that participants experienced few difficulties when creating and executing test cases within the integrated MERLIN environment. In contrast, the statement regarding the understandability of the feedback messages received the highest mean score (3.0), making it the lowest-rated usability aspect. This suggests that, although participants were generally able to interpret the test results, the current presentation of error messages could be improved. This observation is further supported by the qualitative feedback discussed in Section 2.3, where several participants suggested providing more descriptive and less technical error messages.

The questionnaire results indicate that participants perceived the MUnit DSL as sufficiently expressive for specifying both structural and behavioural tests of MERODE models. As shown in Figure 6, all expressiveness-related statements received positive ratings, with mean scores ranging from 1.6 to 1.8 on a five-point Likert scale (1 = Strongly agree, 5 = Strongly disagree). Participants agreed that the DSL adequately supports testing the EDG structure (mean = 1.6) and FSM transitions (mean = 1.6). Likewise, participants indicated that they were able to express expected model behaviour clearly (mean = 1.6).

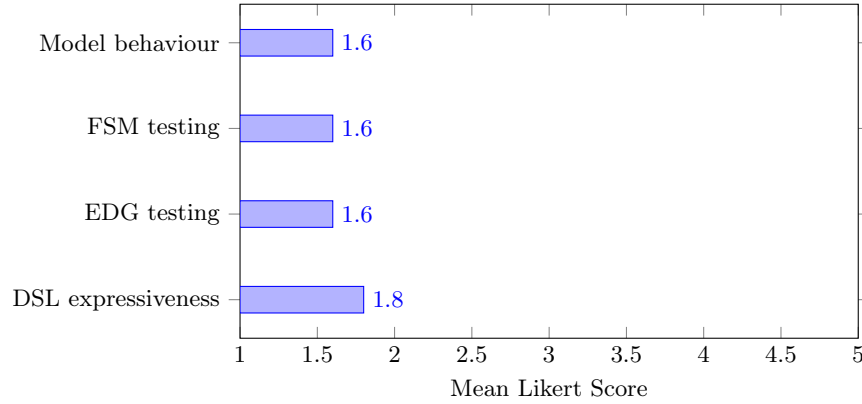


Fig. 6: Mean expressiveness ratings (1 = Strongly agree, 5 = Strongly disagree). The abbreviated labels are listed in Table 3.

The statement *"The DSL allowed me to express relevant MERODE test scenarios"* received a slightly higher mean score (1.8) than the remaining statements. Nevertheless, participants still agreed that the DSL provides sufficient language constructs for modelling relevant testing scenarios. Overall, the questionnaire responses indicate that participants considered the language expressive enough to formulate both structural assertions and behavioural test cases required for testing MERODE models.

The questionnaire results indicate that participants perceived the MUnit testing approach as beneficial for teaching and learning MERODE. As shown in Figure 7, all educational value-related statements received mean scores ranging from 1.2 to 1.8 on a five-point Likert scale (1 = Strongly agree, 5 = Strongly disagree). The statement *"Automated testing could improve students' understanding of MERODE"* received the strongest agreement (mean = 1.2), indicating that participants considered automated testing to have considerable educational potential. Similarly, participants agreed that test-driven modeling could help students detect modeling errors earlier (mean = 1.4) and that the overall approach could improve model quality in educational settings (mean = 1.4).

The statement regarding the learning benefits of immediate feedback received the highest mean score within this category (1.8), although participants still

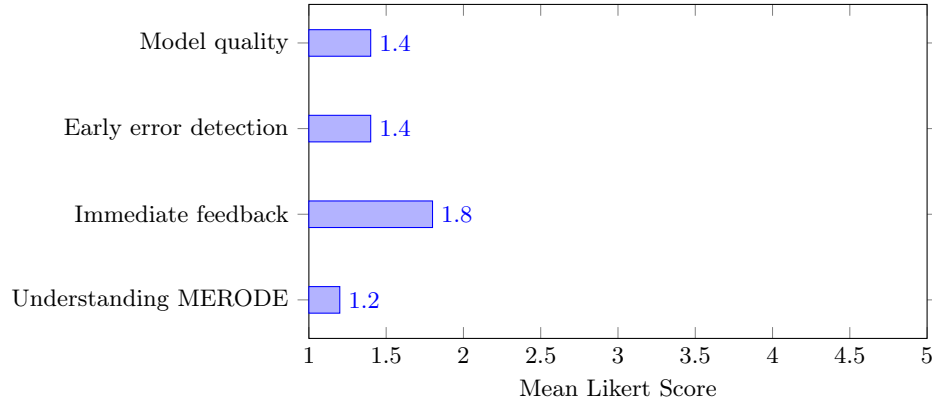


Fig. 7: Mean educational value ratings (1 = Strongly agree, 5 = Strongly disagree). The abbreviated labels are listed in Table 3.

generally agreed that immediate feedback supports learning. Overall, the questionnaire responses suggest that participants perceived automated testing and test-driven modeling as valuable additions to MERODE education, particularly for improving conceptual understanding, supporting early error detection, and enhancing model quality.

The questionnaire results indicate that participants perceived the MUnit testing framework as suitable for integration into MERODE courses. As shown in Figure 8, all course integration-related statements received positive ratings, with mean scores ranging from 1.2 to 2.0 on a five-point Likert scale (1 = Strongly agree, 5 = Strongly disagree). The statement *"I could imagine integrating this tool into MERODE exercises"* received the strongest agreement (mean = 1.2), suggesting that participants considered the proposed approach well suited for practical teaching activities. Similarly, participants agreed that the tool fits naturally into existing MERODE teaching workflows (mean = 1.4).

The statements regarding the tool's potential to support assignment evaluation (mean = 1.8) and improve student engagement (mean = 2.0) received slightly higher mean scores than the remaining statements. Nevertheless, participants generally agreed that the framework could contribute positively to MERODE teaching. Overall, the questionnaire responses suggest that participants perceived the proposed testing approach as a practical addition to existing MERODE courses, particularly for supporting exercises and integrating into established teaching workflows.

Qualitative Findings In addition to the quantitative questionnaire results, qualitative feedback was collected through observations made during the evaluation sessions and participants' responses to the open-ended questions in the questionnaire. The qualitative analysis aimed to identify recurring themes regarding the strengths of the proposed testing framework, challenges encountered while

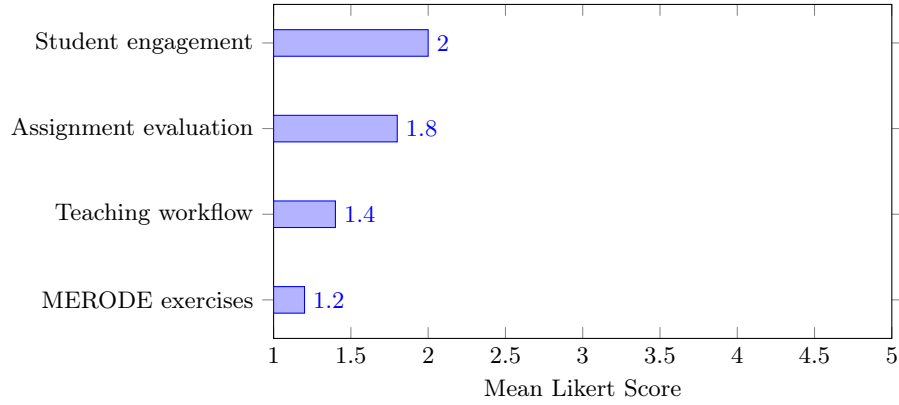


Fig. 8: Mean course integration ratings (1 = Strongly agree, 5 = Strongly disagree). The abbreviated labels are listed in Table 3.

using it, and suggestions for future improvements. Rather than presenting the feedback participant by participant, the comments were grouped into common themes that emerged across the evaluation sessions. Table 5 provides an overview of the identified themes and the aspects discussed within each category.

Table 5: Overview of themes identified during the qualitative analysis.

Theme	Summary of observations
Perceived Benefits	Positive aspects of the proposed approach, including rapid model validation, immediate feedback, reusable test scripts, and the ability to test models without generating executable code.
Language Design	Feedback regarding the DSL syntax and terminology, including naming of language constructs, parameter ordering, case sensitivity, and the required level of programming knowledge.
Editor Support	Suggestions for improving the editor through auto-completion, syntax validation, semantic checking, method suggestions, and visualization of available model elements.
Error Reporting	Recommendations for improving error messages by providing clearer explanations, line numbers, better traceability, and more informative diagnostics.
Additional Feature Requests	Additional suggestions such as undo functionality, support for multiple test scripts, and extensions to existing assertion types.

Perceived Benefits Participants generally expressed a positive opinion of the proposed MUnit testing framework and considered it a valuable extension to the MERLIN modeling environment. A recurring theme throughout the evaluation

was the ability to execute tests directly on MERODE models without first generating executable application code. Multiple participants highlighted that this significantly shortens the feedback cycle during modeling, allowing models to be validated immediately after making changes rather than waiting for the code generation process. Participants described this immediate availability of testing as one of the most valuable aspects of the proposed approach.

Several participants also appreciated the interactive nature of the testing workflow. They described the framework as supporting a rapid trial-and-error process in which models can be modified, tested, and refined. The immediate execution of test cases encouraged frequent validation throughout the modeling process and made experimenting with different modeling decisions straightforward. Participants emphasized that this iterative workflow would enable both students and instructors to identify modeling issues much earlier than in a traditional model-first workflow.

Another frequently mentioned advantage was the seamless integration of test cases into the modeling process. Participants valued that test scripts are automatically synchronized with changes to the model, eliminating the need to manually adapt tests after regenerating code. Furthermore, the ability to save, reuse, and repeatedly execute test scripts was perceived as particularly useful for validating models throughout their development. Short execution times of the test suite was seen as very important, since this allows tests to become a permanent part of the modeling workflow rather than a one-time validation activity.

Beyond these practical advantages, participants also recognized the educational potential of the framework. Several participants believed that allowing students to instantiate models, execute operations, and immediately observe the results would improve their understanding of MERODE concepts and encourage experimentation during the learning process. One participant noted that the framework gives students the opportunity to test their models immediately without regenerating code, while another described the proposed approach as an important first step towards introducing test-driven modeling into MERODE. Similarly, one participant characterized the framework as "*extremely relevant for MERLIN*", highlighting its perceived value for both education and future development of the modeling environment.

Overall, the qualitative feedback suggests that participants perceived the framework not merely as a testing tool but as an integral part of the modeling process. The combination of immediate feedback, rapid iteration, reusable test cases, and educational support was consistently identified as the primary strength of the proposed approach.

Feedback and Suggested Improvements Although participants generally evaluated the proposed testing framework positively, they also identified several opportunities for improvement. Most suggestions focused on improving the usability of the testing language and editor, particularly for inexperienced MERODE users. The recurring themes are summarized below.

- **Language design:** Several participants suggested renaming the `OBJECT` keyword to `OBJECT TYPE` or `CLASS` to better distinguish object types from object instances. In addition, multiple participants proposed reversing the parameter order of dependency assertions by specifying the dependent object before the master object, as this was considered more intuitive.
- **Simplifying the DSL:** Participants noted that some language constructs require knowledge of the generated Java code (e.g., `getState().getName()`) and that the Java-like syntax may be difficult for non-technical students. Furthermore, inconsistent case sensitivity and lowercase method naming were considered confusing, especially for beginners.
- **Editor support:** The most frequently mentioned improvement was the addition of editor assistance. Participants suggested auto-completion for object types, methods, and language keywords, as well as suggestions while typing, buttons for inserting generated methods, visualization of available model instances, and immediate syntax and semantic validation to detect typos or undefined identifiers.
- **Error reporting:** Participants recommended providing more user-friendly error messages by using natural language instead of technical exceptions. Additional suggestions included reporting the failing line number, improving traceability, explaining why assertions failed, reporting expected versus actual values, detecting common mistakes such as typing or case errors, and allowing developers to specify custom assertion messages.
- **Additional features:** Individual participants proposed several further enhancements, including support for multiple test scripts per model, undo functionality, and optional parameters for dependency assertions.